Create Connected Tables

Now we will deal with **connected** data put in different tables.

So, the first step is to create more than one table and connect them, so that each row in one table can reference another row in the other table.

We have been working with heroes in a single table `hero`. Let's now add a table `team`.

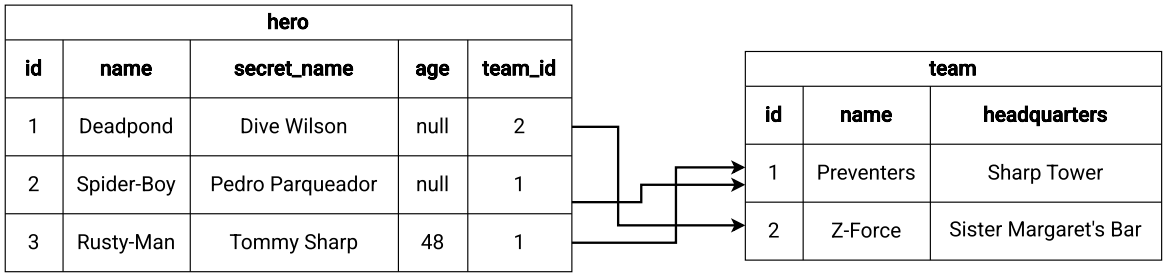
The team table will look like this:

id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

To connect them, we will add another column to the hero table to point to each team by the ID with the `team_id`:

id	name	secret_name	age	team_id ✨
1	Deadpond	Dive Wilson	null	2 ✨
2	Spider-Boy	Pedro Parqueador	null	1 ✨
3	Rusty-Man	Tommy Sharp	48	1 ✨

This way each row in the table `hero` can point to a row in the table `team`:



One-to-Many and Many-to-One

Here we are creating connected data in a relationship where **one** team could have **many** heroes. So it is commonly called a **one-to-many** or **many-to-one** relationship.

The **many-to-one** part can be seen if we start from the heroes, **many** heroes could be part of **one** team.

This is probably the most popular type of relationship, so we'll start with that. But there's also **many-to-many** and **one-to-one** relationships.

Create Tables in Code

Create the `team` Table

Let's start by creating the tables in code.

Import the things we need from `sqlmodel` and create a new `Team` model:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```

This is very similar to what we have been doing with the `Hero` model.

The `Team` model will be in a table automatically named `"team"`, and it will have the columns:

- `id`, the primary key, automatically generated by the database
- `name`, the name of the team

    - We also tell **SQLModel** to create an index for this column
- `headquarters`, the headquarters of the team

And finally we mark it as a table in the config.

Create the New `hero` Table

Now let's create the `hero` table.

This is the same model we have been using up to now, we are just adding the new column `team_id`:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```

Most of that should look familiar:

The column will be named `team_id`. It will be an integer, and it could be `NULL` in the database (or `None` in Python), because there could be some heroes that don't belong to any team.

We add a default of `None` to the `Field()` so we don't have to explicitly pass `team_id=None` when creating a hero.

Now, here's the new part:

In `Field()` we pass the argument `foreign_key="team.id"`. This tells the database that this column `team_id` is a foreign key to the table `team`. A "**foreign key**" just means that this column will have the **key** to identify a row in a **foreign** table.

The value in this column `team_id` will be the same integer that is in some row in the `id` column on the `team` table. That is what connects the two tables.

The Value of `foreign_key`

Notice that the `foreign_key` is a string.

Inside it has the name of the **table**, then a dot, and then the name of the **column**.

This is the name of the **table** in the database, so it is `"team"`, not the name of the **model** class `Team` (with a capital `T`).

If you had a custom table name, you would use that custom table name.

Info

You can learn about setting a custom table name for a model in the Advanced User Guide.

Create the Tables

Now we can add the same code as before to create the engine and the function to create the tables:

Python 3.10+

```
# Code above omitted 👆

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```

And as before, we'll call this function from another function `main()`, and we'll add that function `main()` to the main block of the file:

Python 3.10+

```python
# Code above omitted 👆

def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```

Run the Code

Tip

Before running the code, make sure you delete the file `database.db` to make sure you start from scratch.

If we run the code we have up to now, it will go and create the database file `database.db` and the tables in it we just defined, `team` and `hero`:

```bash
```

Create Tables in SQL

Let's see that same generated SQL code.

As we saw before, those `VARCHAR` columns are converted to `TEXT` in SQLite, which is the database we are using for these experiments.

So, the first SQL could also be written as:

```sql
CREATE TABLE team (
    id INTEGER,
    name TEXT NOT NULL,
```

```
    headquarters TEXT NOT NULL,
    PRIMARY KEY (id)
)
```

And the second table could be written as:

```
CREATE TABLE hero (
    id INTEGER,
    name TEXT NOT NULL,
    secret_name TEXT NOT NULL,
    age INTEGER,
    team_id INTEGER,
    PRIMARY KEY (id),
    FOREIGN KEY(team_id) REFERENCES team (id)
)
```

The only new is the `FOREIGN KEY` line, and as you can see, it tells the database what column in this table is a foreign key (`team_id`), which other (foreign) table it references (`team`) and which column in that table is the key to define which row to connect (`id`).

Feel free to experiment with it in **DB Browser for SQLite**.

Recap

Using **SQLModel**, in most of the cases you only need a field (column) with a `foreign_key` in the `Field()` with a string pointing to another table and column to connect two tables.

Now that we have the tables created and connected, let's create some rows in the next chapter. 🚀